IMPERIAL

Department of Computing
70015: Mathematics for Machine Learning
Lecturer: Matthew Wicker

Lecture 04: Deep Learning & Automatic Differentiation

Additional Resources

- Automatic Differentiation, Section 5.6 (page 159) Deisenroth et al. (2020).
- Online resource with implementation in Rust: https://jingnanshi.com/blog/autodiff.html
- A polished video walk through of automatic differentiation https://www.youtube.com/watch?v=wG_nF1awSSY

4.1 From GLM to the Multi-Layer Perceptron (MLP)

In our last lecture, we saw that for models that are only mildly more complex than neural networks we cannot find a closed form solution for the *"best"* parameter. Here we will begin by recalling the generalized linear model and then build a much more powerful model: the multi-layer perceptron (MLP) which is the simplest neural network model. We will then briefly discuss other neural network architectures and their various uses.

The Perceptron By changing our link function from being denoted g to σ we have that our generalized linear model can be written as:

$$y = \sigma(\theta^\top \boldsymbol{x})$$

Allowing for a basis expansion and using summation notation we can further write:

$$y = \sigma(b + \sum_i \theta_i^\top \boldsymbol{x}_i)$$

This is now exactly the traditional formulation of a single perceptron.

Mutiple Perceptrons However, to form a network of these perceptrons we will need to expand this model. To do so we remove the summation notation, and begin by showing how we can add a second perceptron:

$$[y_0, y_1] = [\sigma(b_0 + \theta_0^\top \boldsymbol{x}), \sigma(b_1 + \theta_1^\top \boldsymbol{x})]$$

Notice that adding a second neuron gave us a second output related to the input by separate parameters vectors. It is notationally clunky to describe things this way so instead we will combine our parameter vectors θ_0 and θ_1 into a matrix: $\theta := [\theta_0, \theta_1]$ such that we have the above equation now as: $\boldsymbol{y} = \sigma(\theta^\top \boldsymbol{x})$ where $\boldsymbol{y} \in \mathbb{R}^2$ and σ is a non-linearity that operates element-wise on the vector $\theta^\top \boldsymbol{x}$. Of course we can add arbitrarily many columns. For example where we have m columns we have a parameter vector $\theta \in \mathbb{R}^{m \times n}$ which when multiplied by our input $\boldsymbol{x} \in \mathbb{R}^{n \times 1}$ gives us our output $\boldsymbol{y} \in \mathbb{R}^{m \times 1}$.

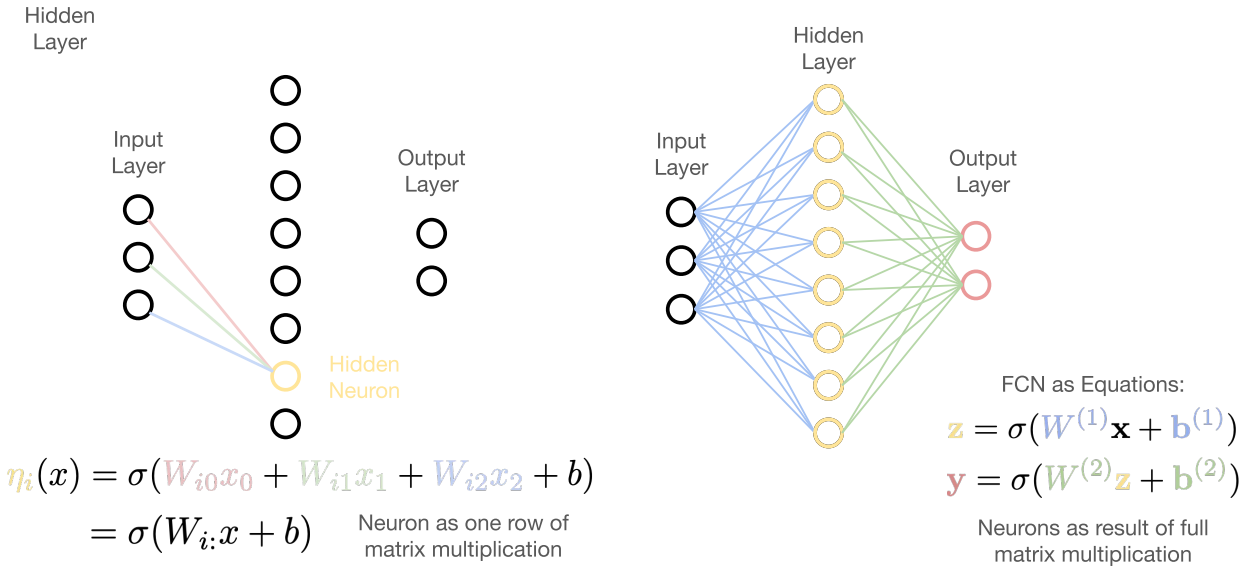


Figure 4.1: Color-coded diagram of a fully-connected neural network model. On the left, we depict a color-coded representation of a single neuron with each input-weight combination having a unique color that matches its representation in the equation. On the right, we depict a color-coded representation of a full neural network with each portion of the equation corresponding to the color of the section in the diagram.

To this point, we have simply shown how to create an n-dimensional generalization of the generalized linear model from our previous lecture. We now discuss how this is turned into a "network" of neurons. For those who have taken a graph theory course, might recall that a $m \times n$ matrix is the canonical representation of the weighted connections in a bipartite graph on m-n nodes where the i, j entry of of the parameter matrix represents the "strength" of the connection between node i (the ith input dimension) in one partition and node j (the jth output dimension) in the other. We visualize some such graphs in Figure 4.1.

The Single-Layer Perceptron By denoting the parameter matrix (previously denoted θ) as $\boldsymbol{W} \in \mathbb{R}^{m \times n}$ we can now introduce the multi-layer perceptron by combining the transformation we have built with the GLM from our previous lecture:

$$y = g^{-1}(\theta^\top \sigma(\boldsymbol{W}\boldsymbol{x} + b)) \tag{4.1}$$

To make the above model more clear we highlight that $\theta \in \mathbb{R}^{m \times 1}$ and break the above equation into two stages:

$$\boldsymbol{z} = \sigma(\boldsymbol{W}\boldsymbol{x} + b) \tag{4.2}$$

$$y = g^{-1}(\theta^\top \boldsymbol{z}) \tag{4.3}$$

The first stage, Equation (4.2), is precisely the hidden neurons we used to describe the bipartite graph. The second stage, Equation (4.3), sends us from what we will call the "hidden layer" of m dimensions in our bipartite graph to the desired output space. This as well can be looked at as a second bipartite graph with weights determined by θ.

The Multi-Layer Perceptron The final step in constructing the multi-layer perceptron is to allow for multiple layers. To do so, we simply add in an additional hidden layer similar to Equation (4.2). The construction of a two-layer perceptron can be given by:

$$\begin{aligned} \boldsymbol{z}^{(1)} &= \sigma(\boldsymbol{W}^{(1)}\boldsymbol{x} + b^{(1)}) \\ \boldsymbol{z}^{(2)} &= \sigma(\boldsymbol{W}^{(2)}\boldsymbol{z}^{(1)} + b^{(2)}) \\ y &= g^{-1}(\theta^\top \boldsymbol{z}^{(2)}) \end{aligned}$$

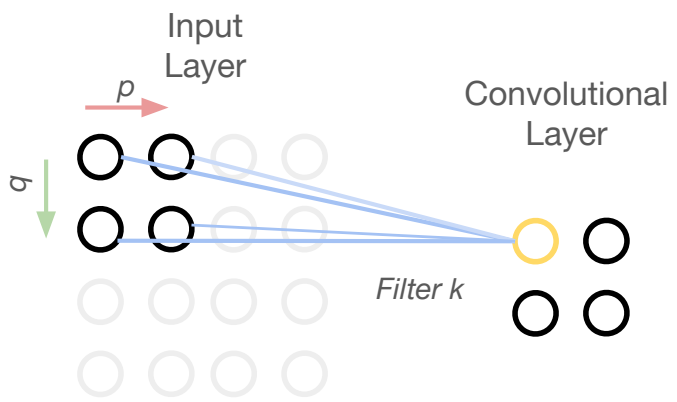
One can then see that generalizing this to an arbitrary number of hidden layers is straight-forward. We ask further questions about this construction in our exercises. We will also provide more rigerous theoretical analysis of this model in future lectures.

4.2 Neural Network Architectures

In this section, unlike the previous, we will not build the layers described from first principles and will instead simply state them as they have been developed and provide some links to further readings.

4.2.1 Convolutional Layers

Convolutional layers are particularly well-suited for processing data that have a grid-like topology, such as images. The main idea behind a convolutional layer is the convolution operation, which involves a kernel or filter that slides over the input data to produce feature maps. Each filter is able to detect different features in the input data, such as edges, textures, or more complex patterns in deeper layers.



$$z_{ijk} = \sigma\left(\sum_{p=1}^{F}\sum_{q=1}^{F} x_{i+p-1,j+q-1} \cdot W_{pqk} + b_k\right)$$

Figure 4.2: A diagram of a single convolutions filter acting on a single channel input with color-coded equation for demonstration purposes. The filter we display is two by two (general: $F \times F$) and the weight contributions (in blue) are multiplied by the input values (in black) and are summed over (red and green).

Formally, given an input $\boldsymbol{x} \in \mathbb{R}^{H \times W \times C}$ (height H, width W, and C channels) and a set of K filters $\boldsymbol{W}_k \in \mathbb{R}^{F \times F \times C}$, the output feature map $\boldsymbol{y} \in \mathbb{R}^{H' \times W' \times K}$ is computed as:

$$y_{ijk} = \sigma\left(\sum_{c=1}^{C}\sum_{p=1}^{F}\sum_{q=1}^{F} x_{i+p-1,j+q-1,c} \cdot W_{pqck} + b_k\right)$$

where σ is a non-linear activation function, and b_k is a bias term. The dimensions H' and W' of the output feature map depend on the stride and padding used in the convolution operation.

Convolutional layers are a cornerstone of Convolutional Neural Networks (CNNs), which have been extremely successful in computer vision tasks such as image classification, object detection, and segmentation.

4.2.2 Recurrent Layers

Recurrent layers are designed to handle sequential data, where the order of data points is important. They are the fundamental building blocks of Recurrent Neural Networks (RNNs), which are used for tasks such as time series prediction, natural language processing, and speech recognition.

In a recurrent layer, the output at each time step depends not only on the current input but also on the hidden state from the previous time step. This allows the network to maintain a memory of past inputs, capturing temporal dependencies.

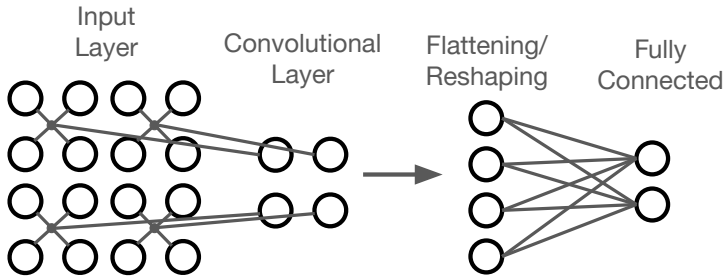


Figure 4.3: A diagram of a simplistic convolutional neural network. The first layer is depicted as a convolution, followed by a flattening transform, followed by a fully connected layer which would represent the output of the model.

Formally, given an input sequence $\boldsymbol{x} = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_T]$, the hidden state $\boldsymbol{h}_t$ at time step t is computed as:

$$\boldsymbol{h}_t = \sigma(\boldsymbol{W}_h \boldsymbol{h}_{t-1} + \boldsymbol{W}_x \boldsymbol{x}_t + \mathbf{b}_h) \tag{4.4}$$

where $\boldsymbol{W}_h$ and $\boldsymbol{W}_x$ are weight matrices, $\mathbf{b}_h$ is a bias term, and σ is a non-linear activation function.

Variants of the basic RNN, such as Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs), have been developed to address issues like vanishing gradients and to better capture long-term dependencies.

4.2.3 Attention Layers

Attention layers have revolutionized many areas of machine learning, particularly natural language processing. The key idea of attention mechanisms is to allow the model to focus on different parts of the input sequence when making predictions, rather than treating all parts of the sequence equally.

Self-attention, or intra-attention, is a common type of attention mechanism where the model computes attention scores within a single sequence. Given an input sequence $\boldsymbol{x} = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_T]$, the attention mechanism computes a set of output vectors $\boldsymbol{y} = [\boldsymbol{y}_1, \boldsymbol{y}_2, \ldots, \boldsymbol{y}_T]$ as:

$$\boldsymbol{y}_i = \sum_{j=1}^{T} \alpha_{ij} \boldsymbol{x}_j$$

where the attention weights α_{ij} are computed using a compatibility function (e.g., dot-product):

$$\alpha_{ij} = \frac{\exp(\boldsymbol{x}_i^\top \boldsymbol{x}_j)}{\sum_{k=1}^{T} \exp(\boldsymbol{x}_i^\top \boldsymbol{x}_k)}$$

Transformers, which rely heavily on self-attention mechanisms, have become the state-of-the-art architecture for many NLP tasks, including machine translation, text generation, and question answering.

For more detailed readings, consider the seminal papers "Convolutional Neural Networks for Visual Recognition" by Yann LeCun, "Long Short-Term Memory" by Hochreiter and Schmidhuber, and "Attention is All You Need" by Vaswani et al.

4.3 Automatic Differentiation

We have already established in the realm of GLMs that computing a gradient and setting it equal to zero was not possible. For neural networks, even computing the gradient is a comlex affair. In fact, for relatively simple neural networks computing gradients by hand is already infeasible. Thus, we would like to derive a systematic approach to differentiation. The approach we will study at is called *automatic differentiation*. We will develop the core concepts and algorithms in automatic differentiation using a small neural network as a running example.

4.3.1 A Running Example

Consider the following one hidden layer neural network that may be used in a regression task:

$$\begin{aligned}
\boldsymbol{z}^{(0)} &= \boldsymbol{x} \\
\zeta^{(1)} &= \boldsymbol{W}^{(1)}\boldsymbol{z}^{(0)} + \mathbf{b}^{(1)} \\
\boldsymbol{z}^{(1)} &= \tanh(\zeta^{(1)}) \\
\zeta^{(2)} &= \boldsymbol{W}^{(2)}\boldsymbol{z}^{(1)} + \mathbf{b}^{(2)} \\
\hat{y} &= \zeta^{(2)} \\
\mathcal{L}(\theta) &= (y - \hat{y})^2
\end{aligned}$$

Now, assuming we have access to i.i.d. samples from our data distribution, we know that we want to find parameters that approximately minimize the loss:

$$\mathcal{L}(\boldsymbol{y}, \boldsymbol{z}^{(2)}) = (\boldsymbol{y} - \boldsymbol{z}^{(2)})^2.$$

Computing $\partial\mathcal{L}/\partial\boldsymbol{W}^{(0)}$, for example, requires computing several intermediate derivatives. For this small network it may be possible to simply expand out the derivative by hand, but for large (e.g., very deep), this is not something that we want to do by hand.

Big picture: The critical idea of automatic differentiation to understand is that we break down any equation (typically constituting our machine learning model) into a set of more primitive operations each of which is simple to differentiate. By understanding the order and composition of these functions, we can write out our model in a way such that we have the output equal to a series of function compositions, e.g.:

$$f(\boldsymbol{x}) = h(g(\boldsymbol{x}))$$

Before, we were interested in an analytical expression for the Jacobian matrix, $\nabla_x f$ that allowed us to solve for our optimal parameters. Assume that $g : \mathbb{R}^n \mapsto \mathbb{R}^k$ and $h : \mathbb{R}^k \mapsto \mathbb{R}^m$, then by chain rule, we have:

$$\boldsymbol{J}_f = \boldsymbol{J}_{h \circ g} = \boldsymbol{J}_h(g(\boldsymbol{x}))\boldsymbol{J}_g(x)$$

Following this general structure we will compute *directional* derivatives from which we can assemble a gradient. Given a series of L compositions of functions we have that the directional derivative computed by automatic differentiation will be:

$$\boldsymbol{J}\boldsymbol{x} = \boldsymbol{J}^{(L)}\boldsymbol{J}^{(L-1)} \dots \boldsymbol{J}^{(1)}\boldsymbol{x} \tag{4.5}$$

Again, we highlight that what we are computing is not the Jacobian matrix itself, but a Jacobian-vector product with respect to the vector $\boldsymbol{x}$ that we call the directional derivative.

4.3.2 Forward-Mode Automatic Differentiation

The first paradigm of automatic differentiation we will cover is forward-mode. The key concept here is that we will compute the necessary derivatives of our function concurrently with value that we compute via the forward pass, defined in Equations (4.2) and Equations (4.3). Here let us highlight that forward mode can be viewed as a way of splitting up Equation (4.5):

$$\begin{aligned}
\boldsymbol{J}\boldsymbol{x} &= \boldsymbol{J}^{(L)}\boldsymbol{J}^{(L-1)} \dots \boldsymbol{J}^{(2)}(\boldsymbol{J}^{(1)}\boldsymbol{x}) && (4.6)\\
&= \boldsymbol{J}^{(L)}\boldsymbol{J}^{(L-1)} \dots \boldsymbol{J}^{(3)}(\boldsymbol{J}^{(2)}\boldsymbol{x}^{(1)}) && (4.7)\\
&= \boldsymbol{J}^{(L)}\boldsymbol{J}^{(L-1)} \dots \boldsymbol{J}^{(4)}(\boldsymbol{J}^{(3)}\boldsymbol{x}^{(2)}) && (4.8)\\
&\dots \\
&= \boldsymbol{J}^{L}\boldsymbol{x}^{(L-1)} && (4.9)
\end{aligned}$$

where the value $\boldsymbol{x}$ with a super-script indicates the last Jacobian matrix we have taken a product with.

Now, let us perform this process on our simple neural network model. When learning neural network parameters we will of course want the gradient w.r.t. the parameters.[1] However, we will walk through computing the input gradient step-by-step. To begin, let us write out the forward pass and error computation of our neural network as two compact equations:

$$\boldsymbol{z}^{(2)} = \text{Matmul}\Big(\boldsymbol{W}^{(1)}, \tanh\big(\text{Matmul}(\boldsymbol{W}^{(0)}\boldsymbol{z}^{(0)})\big)\Big) \tag{4.10}$$

$$\mathcal{L} = (\boldsymbol{y} - \boldsymbol{z}^{(2)})^2 \tag{4.11}$$

The purpose of this step is for us to easily identify which elementary operations are involved in computing the loss. Which will in turn help us write out the computational graph.[2]

[1] If this is practical or useful with forward-mode AD is still up for debate, current research on the matter: https://arxiv.org/pdf/2202.08587.pdf.

[2] Please see the lecture slides for information on constructing a computational graph.

In the automatic differentiation literature these operations make up what are known as *primitives*. We generally want to write down our primitives as taking a small number of arguments (sticking to just two arguments limits mistakes in differentiation). Looking at our equations, we have the following primitives: Matmul, Addition, Tanh, and Pow (raising to a power). For each primitive, we know differentiation rules from calculus. Just to ensure this example is self-contained we include each of them here assuming that we have already computed the value $g(w)$ and have computed the derivative value of $\frac{d}{dw}$.

Operation	**Value Update**	**Derivative Update**
Addition of a constant c	$g(w) + c$	$\frac{d}{dw}(g(w) + c) = \frac{d}{dw}g(w)$
Multiplication by a constant c	$cg(w)$	$\frac{d}{dw}(cg(w)) = c\frac{d}{dw}g(w)$
Raising to a power n	$g(w)^n$	$\frac{d}{dw}(g(w)^n) = n(g(w)^{n-1})\frac{d}{dw}g(w)$
Applying Tanh	$tanh(g(w))$	$\frac{d}{dw}(tanh(g(w))) = 1 - tanh^2(g(w))\frac{d}{dw}$

before doing any computations, lets now fully identify the expressions that we will want to compute:

Value	Derivative
$x_0 = x$	$d_0 = \mathbf{1}$
$x_1 = W^{(0)}x_0$	$d_1 = W^{(0)\top}d_0$
$x_2 = \tanh(x_1)$	$d_2 = 1 - \tanh^2(x_1)d_1$
$x_3 = W^{(1)}x_2$	$d_3 = W^{(1)\top}d_2$
$x_4 = (y - x_3)$	$d_4 = -d_3$
$x_5 = (x_4)^2$	$d_4 = 2(x_4)d_4$

Notice that above we have two columns. One updates the value according to our primitives, thus computing the result of a forward pass through out network. The other updates the derivative with respect to our input such that at the end we have the quantity $d\mathcal{L}/d\boldsymbol{x}$. Now that we have outlined all of our primitives, their derivatives and the computations we want to keep track of we can now compute the value of the derivative (using the parameter values from the beginning). Lets assume that $x = 1, W^{(0)} = 2, W^{(1)} = 1.2$:

$x_0 = 1$	$d_0 = 1$
$x_1 = 1 * 2 = 2$	$d_1 = 2 * d_0 = 2$
$x_2 = \tanh(2) \approx 0.964$	$d_2 = (1 - \tanh^2(2))d_1 \approx 0.14$
$x_3 = 0.964 * 1.2 \approx 1.15$	$d_3 = 1.2 * d_2 \approx 0.169$
$x_4 = (2 - 1.15) \approx 0.85$	$d_4 = -d_3 = -0.169$
$x_5 = (0.85)^2 = 0.722$	$d_5 = 2(0.85)d_4 = -0.283$

Just to clarify what each of the computations above represents, $\boldsymbol{x}_0$ is our input, $\boldsymbol{x}_1$ is the first matrix multiplication, $\boldsymbol{x}_2$ is the application of the activation function, $\boldsymbol{x}_3$ is the second matrix

multiplication, and finally, $\boldsymbol{x}_4$ is the absolute error w.r.t. the label, and $\boldsymbol{x}_5$ is the squared error w.r.t. the label. At the end, we have computed that $d\mathcal{L}/d\boldsymbol{x} = -0.283$. Notice, however that this does not tell us about $d\mathcal{L}/d\boldsymbol{W}_1$ for example. In order to know about this quantity, we would have to perform another forward auto-differentiation. It is potentially a useful exercise beyond what is asked at the bottom of these lecture notes.

4.3.3 Reverse-Mode Automatic Differentiation

In forward mode AD, we saw that we could keep track of both the nominal value of a forward pass as well as the tangents (i.e., the directional derivative). We saw that mathematically a way to motivate this is by associativity of matrix products that give us our Jacobian matrix. Of course, rather than computing the nominal value and the tangents, another perspective is the reverse: compute the adjoints using the nominal values. The adjoint of a variable x can be defined as:

$$\bar{x} = \frac{\partial z}{\partial x}$$

The importance, as we will see below, is that this needs to have the correct shape to be left-multiplied with our Jacobian. Now, using this notation, we can consider an initial output (co-tangent) of our model, e.g., could be the vector of residuals: $\boldsymbol{y} - f(\boldsymbol{x})$. And then proceed in reverse order to compute the numerical derivative w.r.t. the given output. For completeness, we will write out the kind of intuition that we provided for forward mode, but we will see why it might be viewed as misleading (discussion will follow):

$$\begin{aligned}
\bar{\boldsymbol{y}} J &= (\bar{y}\boldsymbol{J}^{(L)})\boldsymbol{J}^{(L-1)} \dots \boldsymbol{J}^{(1)} \\
&= (\bar{\boldsymbol{y}}^{(L)}\boldsymbol{J}^{(L-1)})\boldsymbol{J}^{(L-2)} \dots \boldsymbol{J}^{(1)} \\
&\dots \\
&= \bar{\boldsymbol{y}}^{(2)}\boldsymbol{J}^{(1)}
\end{aligned}$$

Notice that in order to compute things this way in the first place, we need to have $\boldsymbol{y}$ which requires a forward pass through our model. Next, in order to compute the necessary Jacobians, we will need to save our intermediate computation so that we can know precisely what tangent/Jacobian we need to multiply by at each step in the backwards pass. Before jumping directly to the general reverse mode formulation, lets look at the reverse mode computations we would undertake for our neural network. Assume an L-layer fully-connected neural network:

$$\boldsymbol{z}^{(1)} = \boldsymbol{x} \tag{4.12}$$

$$\boldsymbol{z}^{(i+1)} = \sigma(\boldsymbol{W}^{(i)}\boldsymbol{z}^{(i)} + \mathbf{b}^{(i)}) \tag{4.13}$$

Now, lets consider the weight and bias to be joined into parameters denoted $\theta^{(i)} = \{\boldsymbol{W}^{(i)}, \mathbf{b}^{(i)}\}$. Now, we can try to compute the gradient of our function starting from the last layer:

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial \theta^{(L-1)}} &= \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}^{(L)}} \frac{\partial \boldsymbol{z}^{(L)}}{\partial \theta^{(L-1)}} \\
\frac{\partial \mathcal{L}}{\partial \theta^{(L-2)}} &= \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}^{(L)}} \frac{\partial \boldsymbol{z}^{(L)}}{\partial \boldsymbol{z}^{(L-1)}} \frac{\partial \boldsymbol{z}^{(L-1)}}{\partial \theta^{(L-2)}} \\
&\vdots \\
\frac{\partial \mathcal{L}}{\partial \theta^{(1)}} &= \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}^{(L)}} \frac{\partial \boldsymbol{z}^{(L)}}{\partial \boldsymbol{z}^{(L-1)}} \cdots \frac{\partial \boldsymbol{z}^{(3)}}{\partial \boldsymbol{z}^{(2)}} \frac{\partial \boldsymbol{z}^{(2)}}{\partial \theta^{(1)}}
\end{aligned}$$

What we observe is that by the chain rule, we have can compute the partial derivative of the parameters of the ith layer by first computing each of the orange terms which are the partial derivatives of the output of a layer w.r.t. its input and finally multiplying by the blue term which is the derivative of the output of the layer with respect to its parameters.

4.3.4 General Adjoint/Reverse Mode Algorithm

We will now state the general reverse mode automatic differentiation algorithm.[3] Firstly, we will assume that you have a computational graph (we avoid graph theory notation as much as possible in the notes). Let $x_1, \dots x_d$ be the input variables, additionally let $x_{d+1}, \dots x_{D-1}$ be the intermediate variables and finally let x_D be the output variable. Then, we can write out the flow of the computational graph as:

$$\forall i \in [d+1, D] \quad x_i = g_i(x_{\text{Pa}(x_i)}) \tag{4.14}$$

where g_i is the elementary/primitive operation associated with the node in the graph and $\text{Pa}(x_i)$ is the function that returns all of the parents (incoming nodes) to the variable x_i. To compute the derivative of this graph, start by setting the value of derivative of the final variable to 1 (or the vector of ones $\mathbf{1}$). Then, for all other variables apply the chain rule:

$$\frac{\partial f}{\partial x_i} = \sum_{x_j : x_i \in \text{Pa}(x_j)} \frac{\partial f}{\partial x_j} \frac{\partial x_j}{\partial x_i} = \sum_{x_j : x_i \in \text{Pa}(x_j)} \frac{\partial f}{\partial x_j} \frac{\partial g_j}{\partial x_i} \tag{4.15}$$

Let's go ahead and work through an example together. Consider the following equation:

$$f(x) = \sqrt{x^2 + \exp(x^2)} + cos\big(x^2 + exp(x^2)\big)$$

As before, let us break this equation down into its elementary operations or primitives: { square root, exp, power, cos, addition }. From here, we can define the computations that are needed to

[3] This section is taken almost verbatim from MML textbook section 5.6. Please see the surrounding sections if anything here is unclear to you.

evaluate the function:

$$\begin{aligned} x_1 &= x^2 \\ x_2 &= \exp(x_1) \\ x_3 &= x_1 + x_2 \\ x_4 &= \sqrt{x_3} \\ x_5 &= \cos(x_3) \\ x_6 &= x_4 + x_5 \end{aligned}$$

with x_6 being equivalent to the complete function $f(z)$. Now, previously, we would have written down our differentiation rules for each step and then followed the computation through. In reverse mode, I find it is equally useful to write out the derivative of each of the functions before performing our backwards pass:

$$\begin{aligned} \frac{\partial x_1}{\partial x} &= 2x \\ \frac{\partial x_2}{\partial x_1} &= \exp(x_1) \\ \frac{\partial x_3}{\partial x_1} &= 1 = \frac{\partial x_3}{\partial x_2} \\ \frac{\partial x_4}{\partial x_3} &= \frac{1}{2\sqrt{x_3}} \\ \frac{\partial x_5}{\partial x_3} &= -\sin(x_3) \\ \frac{\partial x_5}{\partial x_3} &= 1 = \frac{\partial x_6}{\partial x_5} \end{aligned}$$

Now that we have defined each of the derivatives, we draw a computational graph showing how each of the variables relates to one another via our primitive operations. Once we have written out our computational graph, we work backwards from the output:

$$\begin{aligned} \frac{\partial x_6}{\partial x_3} &= \frac{\partial x_6}{\partial x_4}\frac{\partial x_4}{\partial x_3} + \frac{\partial x_6}{\partial x_5}\frac{\partial x_5}{\partial x_3} \\ \frac{\partial x_6}{\partial x_2} &= \frac{\partial x_6}{\partial x_3}\frac{\partial x_3}{\partial x_2} \\ \frac{\partial x_6}{\partial x_1} &= \frac{\partial x_6}{\partial x_2}\frac{\partial x_2}{\partial x_1} + \frac{\partial x_6}{\partial x_3}\frac{\partial x_3}{\partial x_1} \\ \frac{\partial x_6}{\partial x} &= \frac{\partial x_6}{\partial x_1}\frac{\partial x_1}{\partial x} \end{aligned}$$

When moving backwards through the computational graph, make sure you have accounted (summed) all of the parents as missing a parent will lead to incorrect partial derivatives. Now the above might seem slightly less satisfying when we "compute" it because we have only written out the recipe

for computing the derivative and not the actual value (as we did with forward mode). However, the important aspect here is your understanding of the process as outlined in Equations (4.14) and (4.15).

Learning Objectives and Techniques

- Formulation of feed-forward (fully connected) neural networks
- Hyper-parameters as a concept
- Numerical differentiation as a concept
- Forward-mode automatic differentiation
- Reverse-mode automatic differentiation

Bibliography